

TERNARY SoC EMULATOR

Dokumentacja Techniczna i Architektoniczna – Wersja v1.1

Wersja dokumentu	1.1
Język implementacji	Java 17+ / JavaFX
Architektura procesora	Balanced Ternary, Load/Store
Szerokość słowa	12 trytów (trit-12)
Rozmiar RAM	531 441 komórek (od -265720 do +265720)
Data dokumentacji	02.06.2026

Projekt realizuje pełną emulację 12-trytowego procesora zbalansowanego ternarnego wraz z asemblerem i interaktywnym debuggerem. System operuje na arytmetyce zbalansowanej trójkowej zamiast tradycyjnej binarnej. W wersji v1.1 wprowadzono pełną obsługę skoków zerowych (JMZ).

SPIS TREŚCI

1. Przegląd Projektu
 - 1.1 Cel i zakres
 - 1.2 Architektura systemu – diagram warstw
 - 1.3 Struktura plików i pakietów
2. Podstawy Arytmetyki Zbalansowanej Trójkowej
 - 2.1 Czym jest trit?
 - 2.2 Reprezentacja liczb – zakres i kodowanie
 - 2.3 Właściwości matematyczne
3. Moduł SoC.java – Rdzeń Procesora
 - 3.1 Rejestry sprzętowe
 - 3.2 Organizacja pamięci RAM
 - 3.3 Format słowa instrukcji
 - 3.4 Cykl rozkazowy (fetch-decode-execute)
 - 3.5 Flagi procesora (Status, Overflow, HLT)
 - 3.6 Kompletna lista instrukcji (Dekoder)
4. Moduł Interpreter.java – Asembler
 - 4.1 Faza parsowania i optymalizacji
 - 4.2 Walidacja składni
 - 4.3 Kodowanie instrukcji do RAM
 - 4.4 Dyrektywy asemblera
5. Moduł GUIMain.java – Interfejs Graficzny
 - 5.1 Układ ekranu i panele
 - 5.2 Pasek narzędzi – przyciski akcji
 - 5.3 Edytor kodu źródłowego
 - 5.4 Panel stanu procesora
 - 5.5 Podgląd stosu (Stack Explorer)
 - 5.6 Mapa pamięci RAM
 - 5.7 Dziennik zdarzeń (Event Log)
 - 5.8 Debugger – breakpointy i tryb krokowy
 - 5.9 Wizualizacja graficzna trytów
 - 5.10 Obsługa plików (Open/Save/Save As)
 - 5.11 Okno pomocy (System Manual)
6. Moduł Main.java – Wejście Aplikacji
7. Moduł SystemInfo.java – Informacje Systemowe
8. Instrukcje Asemblera – Pełna Specyfikacja
 - 8.1 Instrukcje arytmetyczno-logiczne
 - 8.2 Instrukcje pamięciowe i rejestrowe
 - 8.3 Operacje stosowe
 - 8.4 Instrukcje sterowania przepływem (w tym JMZ)
 - 8.5 Instrukcje systemowe
9. Przykładowe Programy i Przypadki Użycia
 - 9.1 Hello, Ternary! – proste dodawanie
 - 9.2 Ciąg Fibonacciego
 - 9.3 Obsługa pamięci (Load/Store)
10. Zabezpieczenia i Obsługa Błędów
11. Zależności Techniczne i Wymagania

1. PRZEGLĄD PROJEKTU

1.1 Cel i zakres

Projekt Ternary SoC Emulator to w pełni funkcjonalna emulacja procesora opartego na zbalansowanej arytmetyce trójkowej (ang. Balanced Ternary). W przeciwieństwie do konwencjonalnych komputerów operujących na bitach (wartości 0 i 1), ten procesor operuje na trytach (ang. trits) przyjmujących wartości $\{-1, 0, +1\}$.

Zakres projektu obejmuje:

- Rdzeń procesora (SoC) z pełnym zestawem 27 instrukcji,
- Zbalansowaną pamięć RAM o zakresie adresowym od -265 720 do +265 720,
- Asembler inline tłumaczący tekst mnemoniki na słowa binarne w RAM,
- Graficzny interfejs użytkownika (JavaFX) z debuggerem krokowym,
- Wizualizację rejestrów, stosu i pamięci w postaci kolorowych bloków trytów,
- System breakpointów, dziennika zdarzeń i obsługi plików .txt.

1.2 Architektura systemu – diagram warstw

Warstwa	Komponent	Opis
Prezentacji	GUIMain.java	Interfejs JavaFX: edytor, panele, debugger
Logiki	Interpreter.java	Parsowanie, walidacja, asemblacja do RAM
Sprzętowej	SoC.java	CPU: rejestry, RAM, ALU, cykl rozkazowy
Systemowej	SystemInfo.java	Wersja JVM / JavaFX
Startowej	Main.java	Punkt wejścia (skomentowany CLI), uruchomienie GUI

1.3 Struktura plików i pakietów

Wszystkie klasy należą do pakietu javaproject.

Plik	Klasa	Rola
SoC.java	SoC	Model sprzętowy – CPU + RAM
Interpreter.java	Interpreter	Asembler i interfejs do CPU
GUIMain.java	GUIMain	Główny ekran JavaFX (Application)
Main.java	Main	Skomentowany tryb CLI; wejście aplikacji
SystemInfo.java	SystemInfo	Pomocnicze informacje o środowisku JVM

2. PODSTAWY ARYTMETYKI ZBALANSOWANEJ TRÓJKOWEJ

2.1 Czym jest trit?

Trit (ang. ternary digit) to podstawowa jednostka informacji w systemie trójkowym. Przyjmuje trzy wartości symboliczne:

Symbol	Wartość numeryczna	Kolor w GUI
+	+1 (pozytywny)	Zielony
0	0 (neutralny)	Szary
-	-1 (negatywny)	Czerwony

2.2 Reprezentacja liczb – zakres i kodowanie

Rejestr 12-trytowy reprezentuje wartości całkowite w zakresie $[-265\,720, +265\,720]$. Wartość maksymalna 12 trytów wynika ze wzoru:

$$\text{max} = (3^{12} - 1) / 2 = (531\,441 - 1) / 2 = 265\,720$$

Operacja `truncate12(value)` sprowadza dowolną liczbę całkowitą do powyższego przedziału, realizując arytmetykę modularną:

```
mod = (value + 265720) % 531441
wynik = mod - 265720 (korekta do zakresu symetrycznego)
```

2.3 Właściwości matematyczne

- Symetria wokół zera – nie istnieje problem ze znakiem takim jak w U2.
- Trit środkowy (0) pozwala na naturalne reprezentowanie braku wartości.
- Przesunięcie trytowe (SHF) odpowiada mnożeniu/dzieleniu przez potęgę 3.
- Overflow jest symetryczny: +1 (nadmiar pozytywny) i -1 (nadmiar negatywny).

3. MODUŁ SoC.java – RDZEŃ PROCESORA

Klasa SoC (System on a Chip) jest wirtualnym modelem sprzętowym. Implementuje pamięć RAM, wszystkie rejestry, logikę ALU oraz pełny cykl rozkazowy fetch-decode-execute.

3.1 Rejestry sprzętowe

Nazwa	Rozmiar	Opis	Wartość startowa
PC (Program Counter)	12 trytów	Adres bieżącej instrukcji	START_ADDRESS = -265 720
SP (Stack Pointer)	12 trytów	Wierzchołek stosu w RAM	MAX_VALUE = +265 720
R-	12 trytów	Rejestr ogólnego przeznaczenia (indeks -1)	0
R0	12 trytów	Główny akumulator (indeks 0)	0
R+	12 trytów	Rejestr ogólnego przeznaczenia (indeks +1)	0
STS (Status Flag)	1 tryt	Wynik ostatniej operacji: -1/0/+1	0
OVR (Overflow Flag)	1 tryt	Nadmiar operacji: -1/0/+1	0
HLT (Halt)	boolean	Flaga zatrzymania procesora	false

3.2 Organizacja pamięci RAM

RAM jest tablicą `int[]` o rozmiarze 531 441 elementów, indeksowaną adresami z zakresu `[-265 720, +265 720]`. Każda komórka RAM przechowuje 6 trytów (jedną połówkę słowa maszynowego).

Przeliczenie adresu logicznego na fizyczny indeks tablicy:

```
index = (address + MAX_VALUE) % ramSize
```

Operacje 12-trytowe (LDR, STR, PSH, POP, CALL, RET) automatycznie zajmują dwie sąsiadujące komórki RAM:

```
writeRam12(addr, val) → RAM[addr] = val/729, RAM[addr+1] = val%729
```

3.3 Format słowa instrukcji

Każda instrukcja zajmuje dwie komórki RAM (2×6 trytów = 12 trytów łącznie):

Komórka	Zawartość	Opis bitowy
RAM[PC]	Command Word	[0][Opcode(3T)][Rd(1T)][Rs(1T)] – 6 trytów
RAM[PC+1]	IMM Data	Natychmiastowe dane (6 trytów, zakres <code>[-364, +364]</code>)

Command Word jest kodowany jako: `cmdWord = opcodeId*9 + rd*3 + rs`, gdzie `opcodeId` $\in [-13, +13]$, `rd` $\in \{-1, 0, +1\}$, `rs` $\in \{1, 0, +1\}$.

3.4 Cykl rozkazowy (fetch-decode-execute)

Faza	Operacja	Kod (uproszczony)
1. Fetch	Odczyt słowa komendy z RAM[PC]	cmdWord = readRam(PC)
2. Fetch	Odczyt danych natychmiastowych RAM[PC+1]	imm = readRam(PC+1)
3. Advance	Inkrementacja PC o 2	PC += 2
4. Decode	Wydzielenie pól z cmdWord	shifted = cmdWord + 364
4a	Rejestr źródłowy Rs	rs = (shifted%3) - 1
4b	Rejestr docelowy Rd	rd = (shifted/3%3) - 1
4c	Identyfikator opcode	opcodeId = shifted/9%27 - 13
5. Execute	Wywołanie metody instrukcji	executeDecoded(opcode, rd, rs, imm)

3.5 Flagi procesora

Flaga	Typ	Wartości	Aktualizowana przez
STS (Status)	1 trit	+1 = wynik > 0 0 = wynik = 0 -1 = wynik < 0	ADD, SUB, ADC, SBC, CMP, MIN, MAX, INV, SHF
OVR (Overflow)	1 trit	+1 = overflow+ 0 = brak -1 = overflow-	Jak wyżej (updateFlags)
HLT (Halt)	boolean	true = procesor zatrzymany	Instrukcja HLT

3.6 Kompletna lista instrukcji (Dekoder v1.1)

Mapowanie wewnętrzne (Zgodne ze strukturą inicjalizatora statycznego CPU, w v1.1 instrukcja NOP została usunięta, ustępując miejsca rozkazowi JNZ):

Mnemonik	ID	Mnemonik	ID
SUB	-13	SHF	-12
ADD	-11	SBC	-10
CMP	-9	ADC	-8
MIN	-7	INV	-6
MAX	-5	LDR	-4
LLI	-3	LHI	-2
STR	-1	MOV	0
PSH	1	POP	2
PSF	3	POF	4
JMN	5	OVN	6
JMP	7	WSP	8

Mnemonic	ID	Mnemonic	ID
JMZ	9	HLT	10
CALL	11	OVP	12
RET	13	—	—

4. MODUŁ Interpreter.java – ASEMBLER

Klasa Interpreter działa jako warstwa pośrednia między edytorem tekstu a rdzeniem CPU. Odpowiada za parsowanie kodu mnemonicznego, walidację składni oraz załadowanie skompilowanego programu do RAM procesora.

4.1 Faza parsowania i optymalizacji

Metoda statyczna `optimize(String line)` wykonuje:

- Usunięcie komentarzy inline (wszystko po `//`),
- Przycięcie białych znaków z początku i końca linii,
- Normalizację wielokrotnych spacji do pojedynczej (`replaceAll whitespace`).

4.2 Walidacja składni

Metoda statyczna `validateSyntax(String[] parts)` sprawdza liczbę argumentów na podstawie czterech zbiorów (Set):

Zbiór	Instrukcje	Wymagana liczba argumentów
NO_ARG	HLT, PSF, POF, RET	0 (tylko opcode)
RD_ONLY	INV, PSH, POP, JMN, OVN, JMP, WSP, CALL, OVP, JMZ	1 (Rd)
RD_RS	SUB, SHF, ADD, SBC, CMP, ADC, MIN, MAX, LDR, STR, MOV	2 (Rd, Rs)
RD_IMM	LLI, LHI	2 (Rd, IMM)

4.3 Kodowanie instrukcji do RAM

Metoda `assembleAndLoad(List programLines)` iteruje po liniach programu i wykonuje:

1. Wywołanie `optimize()` → normalizacja linii
2. Rozpoznanie dyrektywy adresu `[n]` → ustawienie `currentRamAddress`
3. Wyznaczenie pól: `opcodeId, rd, rs, imm`
4. Obliczenie: `commandWord = opcodeId*9 + rd*3 + rs`
5. `cpu.injectRAM(addr, commandWord) + cpu.injectRAM(addr+1, imm)`
6. `currentRamAddress += 2` (każda instrukcja = 2 komórki)

4.4 Dyrektywy asemlera

Dyrektywa	Składnia	Działanie
Komentarz linii	<code>// tekst</code>	Ignoruje wszystko za <code>//</code>
Komentarz blokowy	<code># tekst</code>	Ignoruje całą linię zaczynającą się od <code>#</code>
Ustawienie adresu	<code>[adres]</code>	Ustawia bieżący adres RAM dla kolejnych instrukcji
Breakpoint	<code>* instrukcja</code>	Oznacza instrukcję jako punkt zatrzymania debuggera

5. MODUŁ GUIMain.java – INTERFEJS GRAFICZNY

Klasa GUIMain rozszerza javafx.application.Application i stanowi główny punkt uruchomienia graficznego emulatora. Implementuje pełny interfejs użytkownika oparty na ciemnym motywie inspirowanym środowiskiem GitHub.

5.1 Układ ekranu i panele

Panel	Pozycja	Zawartość
Pasek narzędzi (Toolbar)	Góra (top)	Przyciski akcji + wskaźnik statusu
Lewy panel (Left)	Lewa kolumna	Processor State + Stack Explorer
Środkowy panel (Mid)	Środek	Memory Map (RAM Viewer)
Prawy panel (Right)	Prawa kolumna (rozszerzalna)	Edytor kodu + Event Log

5.2 Pasek narzędzi – przyciski akcji

Przycisk	Skrót	Działanie
☐ RESTART	–	Tworzy nowe instancje SoC i Interpreter; czyści log
? MANUAL	–	Otwiera modal z dokumentacją wbudowaną (4 zakładki)
☐ OPEN	Ctrl+O	FileChooser: wczytuje plik .txt do edytora
☐ SAVE	Ctrl+S	Zapisuje do bieżącego pliku lub wywołuje Save As
☐ (dropdown)	–	ContextMenu z opcją 'Save As...' (Ctrl+Shift+S)
☐ SYNTAX CHECK	–	Uruchamia asembler statyczny; walidacja bez wykonania
☐ STEP	–	Wykonuje jedną instrukcję; aktywny po SYNTAX CHECK
☐ RUN FULL	–	Wykonuje program do HLT lub breakpointu (max 1500 kroków)

5.3 Edytor kodu źródłowego

Prawy panel zawiera edytor tekstowy (TextArea). W wersji v1.1 uproszczono podpowiedź tekstową (hintLabel) wyświetlaną, gdy edytor jest pusty, usuwając rozbudowane komentarze instruktażowe na rzecz minimalistycznego szablonu:

```
LLI R0 10 LLI R+ 5 ADD R0 R+ *CMP R0 R+ HLT
```

5.4 Panel stanu procesora (Processor State)

Panel wyświetla aktualny stan wszystkich rejestrów w dwóch formatach: liczbowym oraz graficznym (12 kolorowych blozków trytowych).

5.5 Podgląd stosu (Stack Explorer)

Panel Stack Explorer wyświetla 8 komórek RAM, zaczynając od bieżącego adresu SP w górę. Stos rośnie w dół (SP maleje przy PSH, rośnie przy POP).

5.6 Mapa pamięci RAM (Memory Map)

Środkowy panel wyświetla okno 15 komórek RAM wyśrodkowane na bieżącym PC (od PC-6 do PC+8) wraz z markerem lokalizacji wykonania oraz breakpointów.

5.7 Dziennik zdarzeń (System Event Log)

TextArea na dole prawego panelu rejestrujący błędy kompilacji [ERROR Ln], zakończenie programu [HLT], przepełnienia [KERNEL PANIC] oraz wywołanie punktów zatrzymania [BREAKPOINT].

5.8 Debugger – breakpointy i tryb krokowy

Obsługuje wstrzymywanie maszynowe w stanie runPaused, pozwalając na płynne wykonywanie instrukcji krok po kroku po natrafieniu na oznaczony liniowy adres.

5.11 Okno pomocy (System Manual)

Modal pomocniczy zawierający cztery zakładki (Quick Start, Architecture, Instructions, Examples) zaktualizowany w wersji v1.1 o specyfikację instrukcji skoku zerowego JMZ.

6. MODUŁ `Main.java` – WEJŚCIE APLIKACJI

Klasa `Main` zawiera zakomentowany tryb CLI (wiersz poleceń). Historycznie umożliwiał on ręczne wpisywanie instrukcji w pętli. Obecny kod startowy przekazuje uruchomienie aplikacji bezpośrednio przez wywołanie metody graficznej `GUIMain.main()`.

7. MODUŁ `SystemInfo.java` – INFORMACJE SYSTEMOWE

Klasa pomocnicza udostępniająca statyczne właściwości środowiskowe uruchomionego systemu operacyjnego (wersje JVM oraz JavaFX SDK).

8. INSTRUKCJE ASEMBLERA – PEŁNA SPECYFIKACJA

8.1 Instrukcje arytmetyczno-logiczne

Mnemonic	Składnia	Działanie	Aktualizuje flagi
ADD	ADD Rd Rs	$Rd = Rd + Rs$	STS, OVR
SUB	SUB Rd Rs	$Rd = Rd - Rs$	STS, OVR
ADC	ADC Rd Rs	$Rd = Rd + Rs + OVR$	STS, OVR
SBC	SBC Rd Rs	$Rd = Rd - Rs + OVR$	STS, OVR
CMP	CMP Rd Rs	$STS = \text{sign}(Rd - Rs)$, Rd bez zmian	STS, OVR
MIN	MIN Rd Rs	$Rd = \min(Rd, Rs)$	STS, OVR
MAX	MAX Rd Rs	$Rd = \max(Rd, Rs)$	STS, OVR
INV	INV Rd	$Rd = -Rd$ (inwersja trytu)	STS, OVR
SHF	SHF Rd Rs	$Rd \ll \gg Rs$ trytu (mnożenie/dzielenie 3^Rs)	STS, OVR

8.2 Instrukcje pamięciowe i rejestrowe

Mnemonic	Składnia	Działanie
LLI	LLI Rd IMM	Ładuje IMM do dolnych 6 trytów Rd; zachowuje górne 6
LHI	LHI Rd IMM	Ładuje IMM do górnych 6 trytów Rd; zachowuje dolne 6
LDR	LDR Rd Rs	$Rd = \text{RAM}[Rs]$ (odczyt 12-trytowy z adresu w Rs)
STR	STR Rd Rs	$\text{RAM}[Rs] = Rd$ (zapis 12-trytowy pod adres w Rs)
MOV	MOV Rd Rs	$Rd = Rs$ (kopia rejestru)

8.3 Operacje stosowe

Mnemonic	Składnia	Działanie
PSH	PSH Rd	$SP -= 2$; $\text{RAM}[SP] = Rd$ (12-trytowy push)
POP	POP Rd	$Rd = \text{RAM}[SP]$; $SP += 2$ (12-trytowy pop)
PSF	PSF	Push flag register: $\text{value} = STS * 3 + OVR$
POF	POF	Pop flag register: przywraca STS i OVR
WSP	WSP Rd	$SP = Rd$ (bezpośrednie ustawienie wskaźnika stosu)

8.4 Instrukcje sterowania przepływem

Mnemonic	Syntax	Jump Condition	Action
JMP	JMP Rd	STS > 0 (+1)	PC = Rd (Skok gdy dodatni)
JMN	JMN Rd	STS < 0 (-1)	PC = Rd (Skok gdy ujemny)
JMZ	JMZ Rd	STS == 0 (0)	PC = Rd (Skok gdy zero – Nowość v1.1)
OVP	OVP Rd	OVR > 0 (+1)	PC = Rd
OVN	OVN Rd	OVR < 0 (-1)	PC = Rd
CALL	CALL Rd	bezwarunkowy	PSH PC; PC = Rd
RET	RET	bezwarunkowy	Pop PC (Powrót z RAM Stack)

8.5 Instrukcje systemowe

Mnemonic	Syntax	Action
HLT	HLT	Ustawia flagę HLT=true; zatrzymuje procesor bezpiecznie.

9. PRZYKŁADOWE PROGRAMY I PRZYPADKI UŻYCIA

9.1 Przykład użycia nowej instrukcji JMZ (Skok gdy zero)

Poniższy program odejmuje od siebie dwie równe wartości i wykorzystuje instrukcję JMZ do wykonania skoku warunkowego omijającego nieskończoną pętlę.

```
LLI R0 5 // R0 = 5
LLI R+ 5 // R+ = 5
SUB R0 R+ // R0 = R0 - R+ = 0 (Flaga STS zostaje ustawiona na 0)
LLI R- -265710 // Załaduj adres bezpiecznego wyjścia do R-
JMZ R- // Wykonaj skok jeśli STS == 0 (Nastąpi skok do adresu w R-)
HLT // To się nie wykona
[-265710] HLT // Bezpieczny punkt zatrzymania programu
```

9.2 Obsługa pamięci i adresowania absolutnego []

Przykład zapisu i odczytu wartości z wymuszeniem wskaźnika początku RAM za pomocą dyrektywy nawiasowej:

```
[-250000] // Ustaw wskaźnik alokacji kodu na adres -250000
LLI R0 42 // R0 = 42
LLI R- 250 // Rejestr adresowy R- = 250
STR R0 R- // RAM[250] = R0
LDR R+ R- // R+ = RAM[250]
HLT
```

10. ZABEZPIECZENIA I OBSŁUGA BŁĘDÓW

Zabezpieczenie	Warunek	Reakcja
Limit kroków	stepsExecuted >= 1500 w trybie RUN	Log [TIMEOUT], przerwanie pętli
Stack Overflow	SP <= START_ADDRESS (-265720)	Log [KERNEL PANIC], przerwanie
PC Out of Bounds	PC > 265720 lub PC < -265720	Log [KERNEL PANIC], przerwanie
Nieznany opcode	opcodeId nie w OPCODE_MAP	RuntimeException w SoC.step()
Zła składnia	validateSyntax() rzuca wyjątek	Log [ERROR Ln], pominięcie rzędu

11. ZALEŻNOŚCI TECHNICZNE I WYMAGANIA

Komponent	Wymaganie	Uwagi
Java SE	17 lub nowsza	Środowisko wykonawcze bazowe
JavaFX SDK	21.0.2 (Zgodna z POM)	Wymagane moduły controls i graphics
System OS	Windows / Linux / macOS	W pełni wieloplatformowy przez JVM

Dokumentacja wygenerowana automatycznie dla wersji v1.1. Data: 02.06.2026 | Projekt: Ternary SoC Emulator | Pakiet: javaproject